Technical report — ParlPDFeCH

- **Track:** Track 2A — Extracting Parliamentary Affairs from PDFs into One Common Structure (OpenParlData)
- **Event:** Online
- **Team:** ParlPDFeCH — 1 member (the author)
- **Demo:** no hosted demo. The tool runs on your computer, in the terminal (chapter 7).
- **Repository:** https://github.com/jkovatsch/ParlPDFeCH, folder `track_2a/`.
- **Version:** 16.10.2026. The long version with all tables is docs/report-long.md.

A name in square brackets, for example [`sets.docling.all`], is the id of a number in `numbers.json`. docs/number-ids.md gives the short forms of the ids and their sources. Each link to a file, for example docs/limits.md, goes to the folder `track_2a/` of the repository.

1. Summary

Question: can one method read the 7 head values of a Swiss parliament PDF into an eCH-0295 record, with a probability for each value? The head values are the affair type, title, affair number, submission date, response date, responder and first author.

A PDF goes in. An eCH-0295 record and `<doc>.values.ttl` come out. In `make convert`, each written value has a probability, a page and a text span. Code rules read most values. Apertus-v1.5-8B adds the affair type where the rules find none, and the first author where the rule value is absent or weak. Apertus heads, linear heads on its hidden state, make these 2 choices; Apertus writes no text.

The one fixed run of 13.10.2026 compared the values of 872 [`R.documents`] documents of 26 [`P.all`] parliaments with the OpenParlData export, one time. None is a development parliament. The share correct is the share of documents with a reference that get the export value. For the affair type it is 0.7923 [`R.affair_type.share_correct`] with rules only and 0.8675 [`C.affair_type.share_correct`] with Apertus. For the first author it goes from 0.4556 [`R.first_author.share_correct`] to 0.6028 [`C.first_author.share_correct`]; the responder has 0.8929 [`R.responder_name.share_correct`]. For the affair number, the 2 dates and the title it is less than half (Table 2). The reference is a register, not a gold set, and no pass limit was fixed before the run (chapter 6). No value of this report comes from a test document.

Insight and breadth (chapter 6): eCH-0295 states no condition by affair type: 22 [`cascade.rules.project_choice`] of the 53 [`cascade.rules`] cascade rules are choices of the project. The median precision of a parliament is 0.9841 [`S.affair_type.median_precision`] for the affair type, but 0.5228 [`S.affair_number.median_precision`] for the affair number. The run has 26 parliaments. The 2,665 [`sets.docling.all`] documents are in German, French and Italian, with few exceptions (chapter 4). The cascade has 7 [`cascade.kinds`] kinds of document. Chapter 6 also answers one cross-parliament question, on development documents.

2. Architecture

Input: one PDF. Output: an eCH-0295 record in Turtle and YAML, a label file, `<doc>.values.ttl` and a validation result.

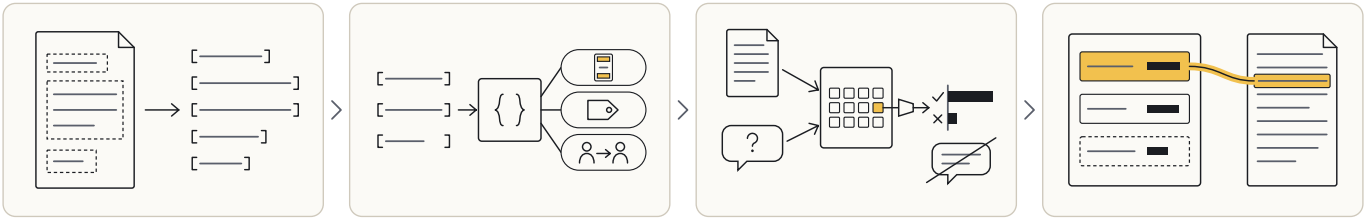


Figure 1: The 4 steps of `make convert`, from left to right; items 1 to 4 below are the 4 panels. Code rules mark page furniture, roles and author changes. In the Apertus step, a linear head gives a probability for each option, and no text comes out. In the record, a yellow marker joins a value to its line on the page; a dashed frame marks an unsure value.

1. **Text blocks.** Docling gives blocks with page, frame and text; code cuts them into lines. On macOS, `ocrmac` reads scanned pages.
2. **Rules.** A rule chain gives the role of the document and of each block, from the PDF alone. The kind of the document (7 options) and the affair type select the slots of eCH-0295 that a document can state. Rule readers read the head values, each with its page and character span.
3. **Apertus step** (chapter 3). With no GPU, it does not run.
4. **Writer and validator.** Each value gets the probability of its tier: on 1,793 [`D.documents`] development documents, the Jeffreys mean of the precision of that tier. A value is certain only if its tier has a precision of 0.95 [`F.addendum_g.certain_tier_precision`] or more over 30 [`F.addendum_g.certain_tier_units`] or more units. Its probability must also reach the threshold of its field. A value below 0.5 [`F.writer_floor`] goes into the

label file only. The record holds the values; <doc>.values.ttl (RDF) and the label file give each its probability, status, page and span. A validator checks the files.

Rules come first: a rule value has a page, a span and a reason that a reader can examine, and it needs no GPU.

3. Use of Apertus

- **Model:** swiss-ai/Apertus-v1.5-8B (form value 8B v1.5), its text branch, bf16, no quantization. No weight value changed.
- **How it is used:** read-out of the hidden state after layer 20 [F.addendum_i.layer_of_each_head] at the last token of each question (Figure 2). Apertus generates no token.
- **Where it runs:** in the run, one H100 GPU made the states, and a Mac applied the heads. make convert uses MLX, torch CUDA, torch MPS or the GPU of another computer.
- **Part of the result:** the affair type and the first author (Table 1).

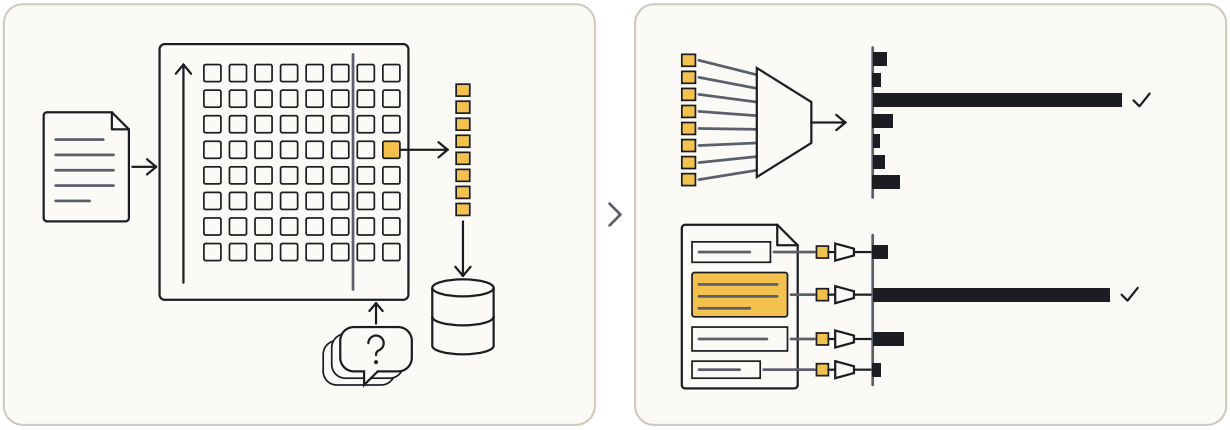


Figure 2: Left: Apertus reads the whole document one time, then each question in its own pass. In the grid, layers go up and tokens go right; the columns right of the grey line are the question. Code keeps the yellow cell, the hidden state after layer 20 [F.addendum_i.layer_of_each_head] at the last token, in a drum. Right: a linear head gives a probability to each option (top); a Q2 head gives one to each block (bottom). Only the heads were fitted.

The German prompt gj_values_v1_de (src/prompts/) has the whole document, one line for each block, then one question. No question sees another. The questions are Q0 (kind of the document), Q1 (affair type) and Q2 (which head values a block states).

The written plan of the run fixed on development documents which questions enter. Q0 did not. Its entry limit was 0.90 and 50 [F.addendum_i.q0_lowest_accuracy_and_documents]: the lowest accuracy and the lowest number of documents. On 140 [H.entry.q0.documents] documents the Q0 head gave 106 [H.entry.q0.head_correct] correct kinds, and the rules also gave 106 [H.entry.q0.rule_correct]. Q1 entered, and of Q2 only the first author with a high head probability.

Rules come first (Figure 3). An Apertus head gives the affair type only where the rules find none, with a head probability of 0.80 [F.addendum_i.lowest_class_probability_q1_q0] or more. It gives the first author only where the rule value is absent or below 0.5 [F.combined_reader_weak_below], with a block probability of 0.80 [F.addendum_i.band_limit] or more. Each tier gives its values one probability from the calibration (Table 1).

Table 1: the 2 Apertus tiers. Stems C.tier.<tier> (run; keys correct, units, precision, stored_probability) and H.tier.<tier> (development, entry data of addendum I; keys equal, different, precision). Reference: the export.

Tier	Correct / units, run	Precision, run	Equal / different, development	Precision, development	Probability from the calibration
apertus_q1 (affair type)	55 / 59	0.9322	93 / 7	0.9300	0.9257
apertus_q2_sponsor_hi (first author)	67 / 79	0.8481	15 / 9	0.6250	0.6200

In the run, the first-author tier replaced 6 [C.tier.apertus_q2_sponsor_hi.replaced_rule_values] weak rule values. No code reads LLM_NAME, LLM_BASE_URL and LLM_API_KEY: the method needs hidden states.

Other models. The Docling layout and table models (open weights) make the text blocks. On macOS, Apple Vision OCR through ocrmac (no open weights) reads scanned pages. AI agents of Claude Opus 5.5 (no open weights) wrote the code, the rules, the measures, the audits and this text. They also gave the AI labels of the block roles.

4. Data

- **Source:** the OpenParlData exports of 04.10.2026 and the PDF copies on https://files.openparldata.ch. Licence of the data: CC BY 4.0, “Source: OpenParlData.ch”.

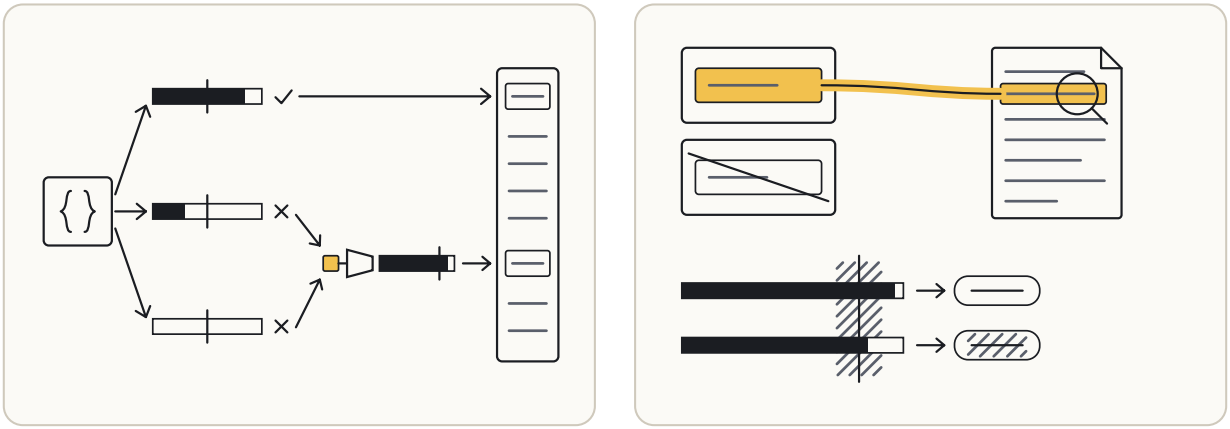


Figure 3: Left: rules first; an Apertus value enters only if it passes its own threshold. Right: in the run, a Q1 type had no page. `make convert` joins it to its type word on the page, or writes no type (crossed-out row). Near a threshold, an Apertus value gets the mark `near` (hatched band).

- **Schema:** eCH-0295, working draft at commit `3cab44d` of https://github.com/swiss/political-affairs-ech-group (version I95). It is not a released standard.
- **Sets, split by parliament:** development, 1,793 documents of 44 [`D.parliaments`] parliaments. The run: 804 [`V.documents`] validation documents of 18 [`P.validation`] parliaments and 68 [`G7.documents`] documents of 8 [`P.group7`] more parliaments (group 7). AI agents had read each group-7 document before the run. A test set of 40 [`sets.test_set.selected`] documents of 13 [`sets.test_set.parliaments`] parliaments was not used.
- **Reference:** the export value after the normalisation of addendum G. For the affair type, it is the eCH type that a project map gives for the export type.
- **Languages** of the 2,665 [`sets.docling.all`] documents: German 1,699, French 676, Italian 268, English 1, none 21 [`sets.language.all.<code>`].
- **Personal data:** names of persons with a public role. The repository has the PDF of the 3 [`pkg.samples.count`] samples only, and no text of a validation, group-7 or test document.

5. Evaluation

The one fixed run. On 13.10.2026 at 21:30, 2 readers gave values for the 872 documents: rules only, and rules with the 2 Apertus tiers. A tool compared each value with the export, one time. Rules, thresholds, heads and entry decisions were fixed before the run (Figure 4).

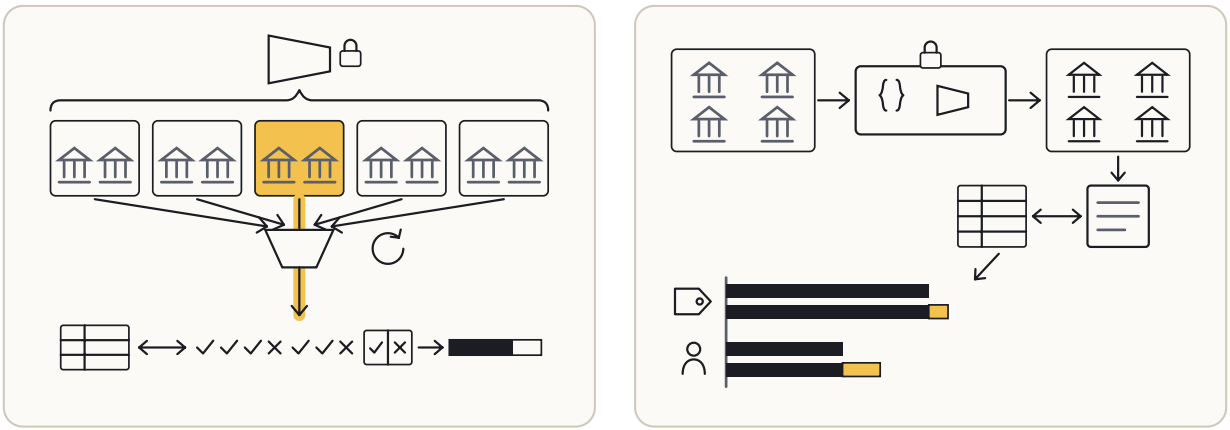


Figure 4: Left: code fitted the heads, not Apertus, on development parliaments in folds; a head fitted on the other folds read each fold (yellow). The share of values equal to the export gives each tier its probability. The run used the final head (lock). Right: the one fixed run on other parliaments; item 5 of chapter 6 gives a limit of this split. Bars: share correct of the affair type (tag) and the first author (person); yellow is the gain of the heads.

Measures. A value is correct if it is equal to the normalised export value, else wrong. Precision is correct / (correct + wrong), found rate is (correct + wrong) / documents with a reference, and share correct is correct / documents with a reference. Each interval is a 90 % interval of a bootstrap by parliament. The 2 response fields count only documents with the rule role `response` or `combined`.

These are measures of the 2 readers of 13.10.2026, not of `make convert`. They count each value, also a value below 0.5 [`F.writer_floor`] that the record does not get. In the run, an Apertus type had no page; `make convert` writes it only where code finds its type word (Figure 3).

Table 2: the one fixed run, by field. Stems R (rules only) and C (with Apertus); intervals `.bootstrap_low5` to `.bootstrap_high95`. Reference: the export. Bar: dark, rules only; yellow, the added share with Apertus; the short line marks half.

Field	With reference	Precision, rules (interval)	Found rate, rules	Share correct, rules (interval)	Share correct, with Apertus	Share correct, bar
affair type	732	0.9431 (0.9139 to 0.9660)	0.8402	0.7923 (0.7213 to 0.8581)	0.8675	
title	866	0.5179 (0.4272 to 0.5972)	0.9053	0.4688 (0.3772 to 0.5573)	0.4688	
affair number	822	0.5275 (0.3493 to 0.6890)	0.4197	0.2214 (0.1358 to 0.3207)	0.2214	
submission date	872	0.5289 (0.4003 to 0.6855)	0.5161	0.2729 (0.2027 to 0.3603)	0.2729	
response date	106	0.5116 (0.3673 to 0.7143)	0.4057	0.2075 (0.1119 to 0.3636)	0.2075	
responder	224	0.9756 (0.9272 to 1.0000)	0.9152	0.8929 (0.7974 to 0.9610)	0.8929	
first author	428	0.8125 (0.7454 to 0.8879)	0.5607	0.4556 (0.3690 to 0.5611)	0.6028	

With Apertus, the found rate of the affair type and of the first author goes up. The precision stays near that of the rules (Table 3). The status certain exists only for the affair type and the responder. The other 5 fields have 0 [`R.title.certain.values`] certain values.

Table 3: found rate and precision with Apertus, and the values with the status certain (rules only). Stems R and C; certain values `R.<field>.certain` (keys `values`, `precision`). Reference: the export.

Field	Found rate, rules	Found rate, with Apertus	Precision, rules	Precision, with Apertus	Certain values	Precision, certain values
affair type	0.8402	0.9208	0.9431	0.9421	460	0.9552
first author	0.5607	0.7313	0.8125	0.8243	0	no value
responder	0.9152	0.9152	0.9756	0.9756	175	0.9714

6. Limitations

Items 1 to 12 are the limits of the result in short form; docs/limits.md gives each in full. Items 13 to 15 are notes on the method.

1. With rules only, fewer than half of the documents with a reference get the export value in 5 of the 7 fields (Table 2).
2. The rule authors read the 44 development parliaments. Addendum I says that they opened no file of the run; nothing compared this with the work records. The found rate is lower in the run: response date 0.4057 [`R.response_date.found_rate`] against 0.9238 [`D.response_date.found_rate`], first author 0.5607 against 0.8488 [`D.first_author.found_rate`].
3. Each of 2 AI examiners named 12 [`E.kind_looks_wrong`] of 50 [`E.documents`] documents of the run with a kind that looks wrong. Where the kind is wrong, the cascade does not search some fields.
4. AI agents had read 116 [`K.read_by_ai_agents.lower_limit`] or more documents of the run before it, with all of group 7. An audit found a sign of earlier contact for 300 [`K.s4.all`] and none for 572 [`K.s3.all`], which is no proof of no contact.
5. AI agents designed the older rule chain (method `nos4`) on 38 [`KD.validation`] validation documents of 9 [`KD.validation_parliaments`] parliaments. For this part, the run is not a measure on new parliaments.
6. AI agents of one model family (Claude Opus 5.5) wrote and examined the rules, the tools, the measures and the audits.
7. The reference is a register. A value can be correct as printed and differ from it (long version, chapter 5.7).
8. No pass limit was fixed before the run, so this report does not say that a field passes.
9. The development values are optimistic: the rule authors read all 44 development parliaments.
10. The reference of the affair type is a project map of the export type. It is also the fit target of Q1.
11. Group 7 (68 documents, 8 parliaments) has wide intervals. There, 5 [`G7.responder_name.wrong`] of 25 [`G7.responder_name.given`] responder values are wrong, all of 1 parliament. The certain responder values of group 7, 21 [`G7.responder_name.certain.values`] values, have a precision of 0.7619 [`G7.responder_name.certain.precision`].
12. Each time comes from one laptop clock, with no outside time stamp.
13. The share correct and the spread between the parliaments were not planned measures of the run.
14. A second tool and a third calculation gave the same counts from the same files. This does not show that a value is true.
15. On 14.10.2026 a private build pipeline on Linux converted the 3 samples with the worker of the package and built a container image. That image was not started.

Insight and breadth

- eCH-0295 states no condition by affair type: 22 of the 53 cascade rules are choices of the project.
- Only a register can give 12 [`cascade.slots.register_only`] of the 87 [`cascade.i95.slots`] slots. The kind of a document selects 18 [`cascade.slots_by_kind.other.low`] or more slots, and at most 61

[cascade.slots_by_kind.minutes.high].
- The median precision of a parliament is 0.9841 for the affair type, but 0.5228 for the affair number. In each of the 7 fields, one parliament or more is at 0.4286 [S.affair_type.lowest_precision] or less. For the response date only 2 [S.response_date.parliaments_that_count] parliaments count. A pooled value does not describe one parliament.
- Rules only, the share correct of the affair type is 0.8109 [V.affair_type.share_correct] on the validation parliaments and 0.5636 [G7.affair_type.share_correct] in group 7. For the first author it is 0.4474 [V.first_author.share_correct] and 0.5088 [G7.first_author.share_correct].
- For 276 [gaps.affair_type.all] of 2,665 documents, the export has no affair type of the I95 list.

Cross-parliament question

In which parliaments does the PDF with the response of the executive also hold the submitted text? The export does not give this answer. On development documents, an Apertus head gives each block a role (B1-S, long version chapter 3.8). With these roles, 300 [xp.written.with_both] of 719 [xp.written.with_response] documents with response text also hold submitted text: 0.417 [xp.written.share_both] (0.387 [xp.written.share_low5] to 0.448 [xp.written.share_high95]). 3 other views give 0.503 [xp.expected_d1.share_both] to 0.548 [xp.d1_label.share_both]. 32 [xp.parliaments_with_statement] of 44 [xp.parliaments] parliaments get a statement. Limits: the roles are measured against labels from an AI agent only, and the labels of 193 [sets.reference.dev.documents] documents are in-sample. The sample is stratified by export role, so a share is not the share of a parliament. This is information, not a result of the one fixed run.

7. Reproducibility

Run the commands in the folder track_2a/ (docs/run-locally.md).

- make or make info: the state of the setup, the GPU and the results. It changes nothing.
- make setup (network): one command for the packages, the PDF reader and the Apertus step. The model needs your own Hugging Face account. The head files are CC-BY-4.0, at a pinned revision. With no GPU, make convert uses the code rules only.
- make gpu: it checks the GPU and names the ways: MLX (a Mac with Apple silicon and 32 GB), torch CUDA (24 GB or more), torch MPS, or the GPU of another computer (make serve-apertus, APERTUS_URL; docs/apertus-gpu.md).
- make convert PDF=<file or URL> (or DIR=, LIST=): the record, <doc>.values.ttl and the validation result in out/<ID>/, and each value with its probability in the terminal. make view ID=<ID> shows one result in the browser.
- make test: checks with no network and no model. It has no check of the numbers.
- make run (Docker) shows the 3 stored sample results only. It runs no Apertus.
- **Tested** on 16.10.2026, only on one Mac (Apple M4, 32 GB, MLX). On 31 development documents, the Apertus step gave the same decisions as the stored H100 states. Not tested: the model download, a CUDA GPU with this code, Linux, a second computer, and make setup with no GPU.
- **Hardware of the run:** an Apple M4 (readers, measures, validation) and one H100 GPU (states). The states, the fit data and the export rows are not in the repository.

8. Next steps

1. A gold set: reference values and the kind of the document for a sample of documents.
2. A measure of the kind against labels; value rules for more parliaments; a run on documents that no agent read.
3. The Apertus step on a CUDA GPU and on Linux; a start of the container image on Linux.

Licence

Copyright 2026 the author (see NOTICE). This report and the documentation: CC-BY-4.0. Code and prompt files: Apache-2.0. OpenParlData data: CC BY 4.0. Apertus: Apache-2.0 with the Apertus Acceptable Use Policy v1.5. Hack Apertus terms, chapter 6: https://hackapertus.ch/terms-and-conditions

References

1. Challenge: https://hackapertus.notion.site/track-2a-openparldata. OpenParlData: https://openparldata.ch
2. eCH-0295 working draft: https://github.com/swiss/political-affairs-ech-group (commit 3cab44d)
3. Apertus-v1.5-8B: https://huggingface.co/swiss-ai/Apertus-v1.5-8B. Use policy: https://github.com/swiss-ai/apertus-legal/blob/main/apertus_1.5/USAGE_POLICY.pdf
4. Docling: https://github.com/docling-project/docling. Read-out principle (Jev): https://openjev.com
5. ASD-STE100 Simplified Technical English: https://asd-ste100.org

About this text: AI agents (Claude Opus 5.5) wrote it from the records of the private project, in English to the rules of ASD-STE100 (Issue 9). On 16.10.2026 an AI agent made it shorter and added the figures.